

Adaptive Tracing Across Four Microservice Systems

Comparative Evaluation of Controller Families, Runtime-Change Behavior, and Taxonomy-Guided Interpretation

Spandan Bhattarai

June 22, 2026

Abstract

This paper presents a comparative study of adaptive tracing across four microservice applications: Train-Ticket, the Timescale OpenTelemetry Demo, Spring Petclinic Microservices, and Grafana QuickPizza. The study has three analytical objectives: first, to compare reinforcement-learning and non-reinforcement-learning tracing controllers under matched operating conditions; second, to examine how reinforcement-learning controllers behave under targeted runtime changes such as latency spikes, error bursts, and throughput drops; and third, to synthesize the implemented methods into a practical taxonomy for engineering use. The cross-system results show that no controller dominates across all environments. Q-Learning is the strongest overall method on Train-Ticket, the rule-based baseline gives the best final latency on the Timescale application, Bandit is the strongest Petclinic policy on final runtime metrics, and the QuickPizza results show that the same controller families remain usable on a Grafana-centered observability stack. Under runtime-change scenarios, Bandit wins the most latency cases overall, while SARSA wins the most throughput cases. The main engineering conclusion is that controller quality is system-dependent, and adaptive tracing should be selected according to runtime stability, operational transparency requirements, and the quality of the feedback signals available for control.

1 Introduction

Distributed tracing is a core observability mechanism for microservice applications, but a fixed tracing configuration imposes a persistent tradeoff. Higher sampling improves diagnostic visibility at the cost of additional telemetry volume, while lower sampling reduces overhead but weakens failure analysis and performance diagnosis. Adaptive tracing addresses this tension by modifying the tracing rate at runtime in response to observed system behavior. The study is grounded in established distributed tracing literature and platforms, including Dapper [1], X-Trace [2], Canopy [3], Pivot Tracing [4], OpenTelemetry [11], Jaeger [12], and Zipkin [13], together with project readings on adaptive observability and RL-guided logging [14].

The practical question is not whether adaptation is useful in principle, but which form of adaptation is strongest under realistic operating conditions. This paper therefore treats adaptive tracing as a control problem and evaluates multiple controller families rather than a single adaptive algorithm. The resulting contribution combines a four-system comparison, a runtime-change study, and a method taxonomy grounded in implemented experiments rather than only conceptual classification.

1.1 Practical usefulness

This study is intended to support a practical decision that observability teams face in production systems: how to retain diagnostically useful traces without paying unnecessary telemetry cost under changing workload conditions. The value of the work is therefore not only that it compares algorithms, but that it translates the comparison into controller-selection guidance.

In practical terms, the paper helps answer three operational questions: when a simple transparent heuristic is sufficient, when reinforcement learning becomes worthwhile, and how much the answer depends on the runtime behavior of the application itself. That practical usefulness is central to the interpretation of the results.

2 Study Objectives and Scope

The paper is organized around three analytical objectives:

- Objective A:** compare reinforcement-learning and non-reinforcement-learning adaptive tracing controllers under matched system conditions;
- Objective B:** evaluate how reinforcement-learning controllers behave under latency spikes, error bursts, and throughput drops; and
- Objective C:** derive a practical taxonomy of adaptive tracing strategies based on the completed experiments.

The completed evidence base consists of 40 controller-comparison runs, 2 fixed-rate baselines, and 48 runtime-change reinforcement-learning runs across the four applications. Grafana QuickPizza contributes 10 controller-comparison runs and 12 runtime-change runs within that total, alongside 3 fixed-rate checks used as supporting context.

3 Experimental Systems

The four experimental applications were selected from established open-source microservice demo and benchmark projects: Train-Ticket [5], the OpenTelemetry Demo / Astronomy Shop [6, 7], Spring Petclinic Microservices [8, 9], and Grafana QuickPizza [10]. These project sources were used during system selection, deployment, instrumentation review, and experimental setup.

4 Methodology

4.1 Adaptive tracing controllers

The study evaluates five adaptive controllers. Q-Learning and SARSA are value-learning reinforcement-learning methods. Bandit is a simpler action-value controller with weaker assumptions about longer-term dynamics. Rule is a threshold-based heuristic baseline. K-Means is a clustering-based baseline that maps recent runtime states to representative sampling rates. The common action space is a discrete set of tracing rates: 0.05, 0.10, 0.20, 0.50, and 0.80.

4.2 Measured outputs

The primary outputs are latency, throughput (QPS), error rate, and trace totals. For reinforcement-learning methods, average reward is also reported. Reward is useful for RL-internal interpretation, but it is not treated as a universally comparable system-quality score because reward design differs across the applications, especially in the Timescale system. Data quality is tracked through decision counts and zero-window counts, because sparse windows weaken the reliability of adaptive-control conclusions.

4.3 Study design

4.4 Methodological caveats

- Reward is reported as contextual reinforcement-learning evidence, not as a uniformly comparable cross-system winner metric.
- The applications are not symmetric in evidence quality. Train-Ticket is the strongest result set, Timescale is valid but has reward-design caveats, and Petclinic is complete but much sparser.
- Fault injection mechanisms differ by application because the systems expose different operational controls. The study therefore emphasizes within-system comparisons first and cross-system synthesis second.
- QuickPizza uses the same controller families and scenario structure as the other systems, but its observability stack and service topology are distinct, so its results should still be interpreted as a separate system rather than as a direct replica of any other environment.

5 Comparative Evaluation of Controller Families

The cross-system comparison shows that reinforcement-learning controllers are consistently competitive, but there is no universal winner across the four applications. Q-

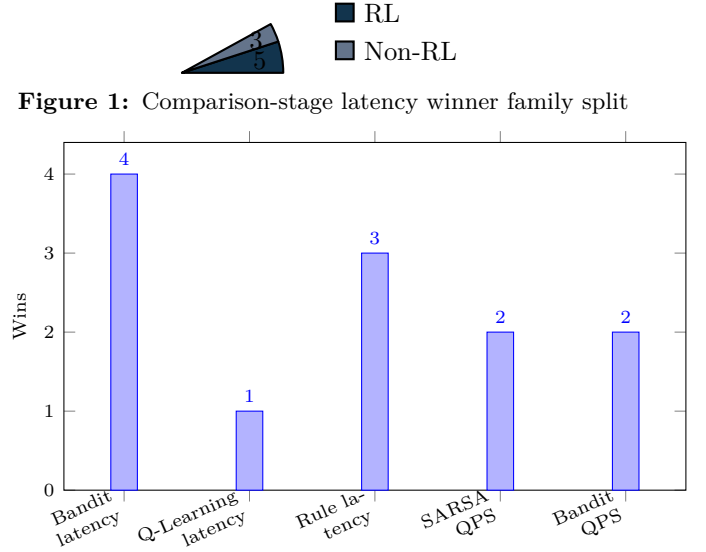


Figure 2: Selected policy win counts in the comparison study

Learning is strongest on Train-Ticket, the rule-based baseline is strongest on Timescale final latency, Bandit is strongest on Petclinic final runtime metrics, and Quick-Pizza shows that the same controller families remain viable on a Grafana-centered stack. The evidence therefore supports a contingent rather than absolute view of controller quality.

Interpretation. Train-Ticket provides the clearest evidence for Q-Learning because it combines strong faulted performance, strong contextual reward, and a mature decision horizon. Timescale shows that a transparent heuristic can still outperform reinforcement learning on final latency, which prevents an overgeneralized conclusion that learning-based control is always superior. Petclinic shows that a simpler reinforcement-learning family member, Bandit, can be preferable when the runtime signal is noisier and operational stability matters more than deeper value learning. QuickPizza extends that comparison onto a Grafana-centered stack and shows that the same controller families remain analyzable there as well.

What these results mean. The main meaning of the controller-comparison results is that complexity alone does not guarantee better adaptive tracing. More sophisticated learning-based control can be highly effective, but its advantage depends on whether the application produces a stable enough feedback signal to support learning. When the signal is cleaner, deeper reinforcement learning can justify its additional complexity. When the signal is less stable, simpler methods may be more reliable and easier to operate.

For practitioners, this means that controller choice should be framed as an engineering-fit problem rather than an algorithm race. The correct question is not which controller won overall, but which controller is most appropriate for the operational characteristics of the target

Table 1: Experimental systems used in the study and their role in the evidence base.

System	Domain	Technical role in the study	Observability / tracing context	Comparative winner
Train-Ticket	Railway ticket booking benchmark	Largest and most stable application in the study; strongest evidence base	Python microservices with adaptive tracing controller and experiment runners	Q-Learning
Timescale OpenTelemetry Demo	Password-generation microservice application	Lightweight tracing-first microservice system with strong observability	OpenTelemetry Collector, Prometheus, TimescaleDB, Jaeger, Grafana	Rule
Spring Petclinic Microservices	Spring Cloud business-domain microservice application	Most realistic business application, but also the noisiest result set	API gateway, service discovery, config server, Zipkin, Prometheus	Bandit
Grafana Quick-Pizza	Pizza recommendation microservice application	Grafana-maintained demo system used as the fourth comparative application in the study	Grafana local stack with Alloy, Prometheus, Tempo, Loki, Pyroscope, Grafana, and QuickPizza microservices	Bandit

Table 2: Design structure by analytical objective.

Objective	Compared methods	Scenario structure	Main outputs
Comparative evaluation	Q-Learning, SARSA, Bandit, Rule, K-Means	Healthy and faulted per system across all four applications	Final latency, final QPS, error rate, contextual RL reward
Runtime-change analysis	Q-Learning, SARSA, Bandit	Healthy, latency spike, error burst, throughput drop across all four applications	Latency/QPS winners per scenario, average selected rate, data quality
Taxonomy synthesis	Taxonomy over implemented methods	Uses completed comparison and runtime-change evidence; no new experiment matrix	Method categories, decision signals, behavior patterns, operational implications

system. The inclusion of QuickPizza as a fourth system reinforces that core conclusion across a broader range of observability stacks.

Table 3: Cross-system summary of controller-comparison winners.

Application	Scenario	Lowest latency	Value	Highest QPS	Value	Contextual RL reward leader	Avg. reward
Train-Ticket	Healthy	Bandit	118.58 ms	SARSA	62.551	Q-Learning	0.1031
Train-Ticket	Faulted	Q-Learning	96.29 ms	Q-Learning	35.360	Q-Learning	0.0767
Timescale Open-Telemetry Demo	Healthy	Rule	133.85 ms	SARSA	2.630	SARSA	0.3706
Timescale Open-Telemetry Demo	Faulted	Rule	211.29 ms	K-Means	2.275	Q-Learning	0.5073
Spring Petclinic Microservices	Healthy	Bandit	15.86 ms	Bandit	0.945	Bandit	0.2105
Spring Petclinic Microservices	Faulted	Bandit	8.11 ms	Bandit	4.418	Bandit	1.3779
Grafana Quick-Pizza	Healthy	Rule	116.28 ms	Rule	9.196	Bandit	1.4693
Grafana Quick-Pizza	Faulted	Bandit	0.00 ms	Q-Learning	8.996	Bandit	2.8662

6 Controller Behavior Under Runtime Changes

Controller behavior under runtime change is not uniform. Across the sixteen system-scenario cases, Bandit wins the most latency cases (7), SARSA wins the most QPS cases (8), and contextual reward leadership is split across all three reinforcement-learning policies. This indicates that different controllers react differently to degradation patterns rather than sharing a common adaptation profile.

Interpretation. Train-Ticket remains the strongest environment for learning-oriented policies because it supplies long decision horizons and stable traffic. Timescale exposes reward-design weakness most clearly; the latency-spike case shows that contextual reward can appear favorable even when latency becomes extreme, so runtime metrics must remain primary. Petclinic remains usable but sparse. Its results still contribute meaningfully, but the short decision horizon makes the claims there weaker than in Train-Ticket.

What these results mean. The runtime-change results show that reinforcement-learning controllers do not share one common adaptation behavior. One method can be better at controlling latency, while another can preserve throughput more effectively under the same disturbance. This means that runtime-change behavior is itself a design criterion, not just a secondary observation.

For operations teams, the implication is direct: controller evaluation should include the kinds of disturbances that actually matter in deployment. A method that looks strong in a stable scenario may not be the right choice if the real concern is bursty slowdown, transient failures, or throughput instability.

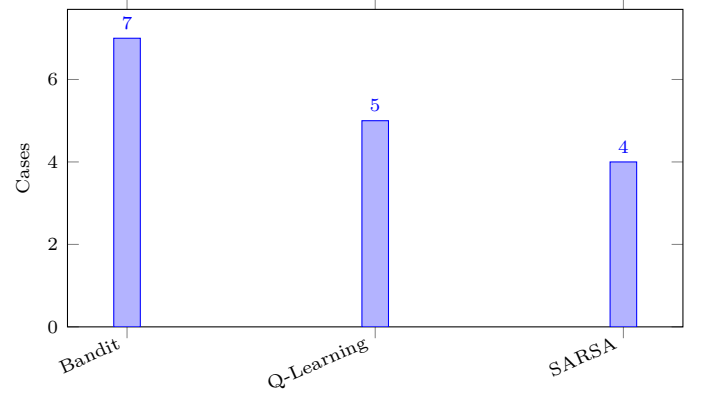
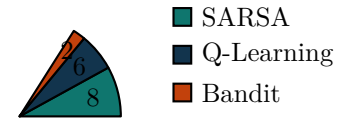
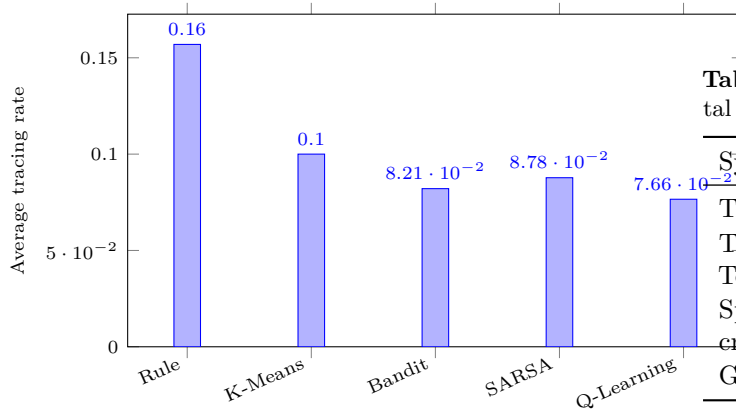
**Figure 3:** Latency wins under runtime-change scenarios**Figure 4:** Runtime-change QPS winner split

Table 4: Cross-system summary of runtime-change winners.

Application	Scenario	Latency winner	Value	QPS winner	Value	Contextual RL reward leader	Avg. reward	Zero windows
Train-Ticket	Healthy	Q-Learning	54.64 ms	SARSA	68.727	Q-Learning	2.2526	0
Train-Ticket	Latency Spike	Q-Learning	73.76 ms	SARSA	34.769	Bandit	0.0405	0
Train-Ticket	Error Burst	Bandit	78.15 ms	Q-Learning	45.003	SARSA	0.0496	0
Train-Ticket	Throughput Drop	Bandit	78.09 ms	SARSA	24.638	Bandit	0.0283	0
Timescale Open-Telemetry Demo	Healthy	SARSA	198.71 ms	SARSA	1.931	Q-Learning	0.4598	0
Timescale Open-Telemetry Demo	Latency Spike	Bandit	344.65 ms	Bandit	1.605	Q-Learning	11.9646	0
Timescale Open-Telemetry Demo	Error Burst	Q-Learning	334.96 ms	SARSA	1.447	SARSA	0.4667	0
Timescale Open-Telemetry Demo	Throughput Drop	SARSA	140.85 ms	SARSA	3.330	SARSA	0.3228	0
Spring Petclinic Microservices	Healthy	SARSA	10.05 ms	Bandit	3.037	Bandit	0.8426	1
Spring Petclinic Microservices	Latency Spike	Bandit	5.23 ms	Q-Learning	7.942	Q-Learning	1.7679	0
Spring Petclinic Microservices	Error Burst	Q-Learning	12.11 ms	SARSA	4.644	SARSA	0.6268	0
Spring Petclinic Microservices	Throughput Drop	Bandit	17.81 ms	Q-Learning	2.519	Q-Learning	0.7116	1
Grafana Quick-Pizza	Healthy	Q-Learning	125.19 ms	Q-Learning	9.000	SARSA	1.4311	2
Grafana Quick-Pizza	Latency Spike	SARSA	124.38 ms	Q-Learning	9.095	Q-Learning	3.0482	3
Grafana Quick-Pizza	Error Burst	Bandit	4.49 ms	Q-Learning	14.401	Bandit	4.5680	5
Grafana Quick-Pizza	Throughput Drop	Bandit	131.98 ms	SARSA	0.900	SARSA	0.1302	0

**Figure 5:** Average selected tracing rate across the full study**Table 5:** System-level data quality across the two experimental blocks.

System	Comparison runs	Comparison avg. decis
Train-Ticket	10	24.3
Timescale Open-Telemetry Demo	10	4.0
Spring Petclinic Microservices	10	1.9
Grafana QuickPizza	10	3.0

7 Cross-System Visual Synthesis

The aggregated patterns become clearer when the study is viewed at the system level rather than only at the policy winner level. A compact data-quality view is therefore useful for explaining why some controller outcomes are more credible or more stable than others.

Interpretation. This system-level view shows why the report should not collapse into a single universal ranking. Train-Ticket supplies the cleanest decision structure, which is why its controller conclusions are the strongest. Timescale is compact but still stable enough to support clear conclusions. Petclinic and QuickPizza remain useful, but their zero-window profiles require more caution when interpreting small differences between methods.

Why this matters. The visual synthesis adds a second layer to the results: not only which controller wins, but how trustworthy the decision process was in each environment. This is operationally important because adaptive tracing quality depends not only on the controller algo-

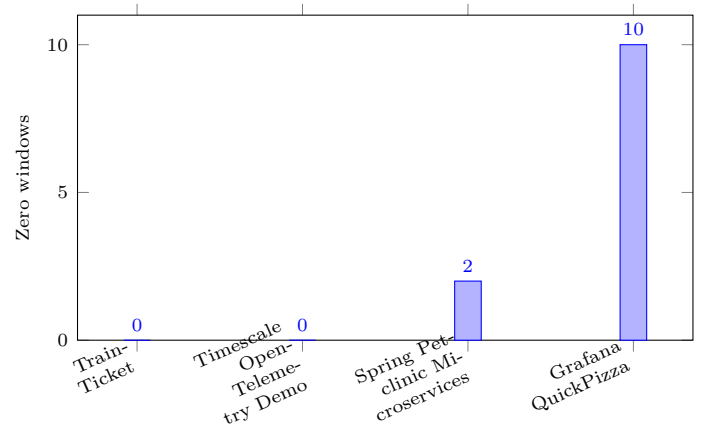


Figure 6: Runtime-change zero-window count by system

rithm, but also on the stability and granularity of the feedback loop that drives it.

8 Taxonomy and Design Guidance

Adaptive tracing is best understood as a family of control strategies rather than one technique. The completed project supports a five-part practical taxonomy: fixed-rate baseline, rule-based control, clustering-based control, bandit-style reinforcement-learning control, and value-learning reinforcement-learning control.

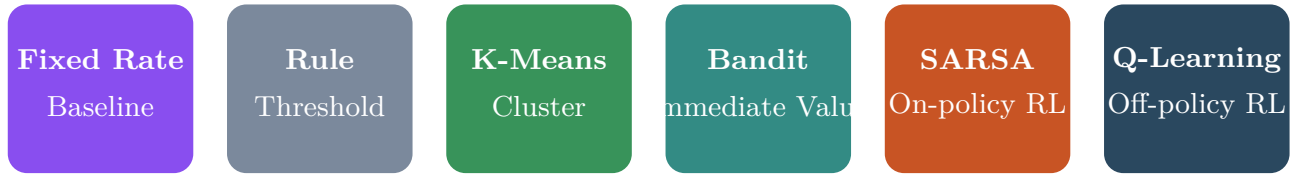
Why the synthesis matters. The completed experiments do not support a single universal controller. Instead, they support a taxonomy in which controller families differ in goals, decision signals, exploration behavior, and operational cost. This is the appropriate abstraction for making deployment decisions in practice.

What these results mean. The taxonomy matters because it converts a long list of experimental observations into a clearer design model. Instead of treating adaptive tracing as a single technique, the paper shows that it should be understood as a family of strategies that trade off interpretability, responsiveness, data requirements, and operational cost in different ways.

This gives the results a practical meaning beyond benchmarking: it provides a vocabulary for selecting, justifying, and communicating adaptive tracing choices in real engineering settings.

Table 6: Practical taxonomy of adaptive tracing strategies.

Method	Type	Main goal	Decision signals	Operational overhead	Observed practical takeaway
Fixed Rate	Static baseline	No runtime adaptation; constant configured rate	Configuration time only	None after startup	Reference point for what is lost without adaptation.
Rule	Heuristic threshold control	Change tracing when thresholds are crossed	Thresholds on latency, errors, counts, or similar runtime signals	Low	Strong when system behavior is predictable and operational transparency matters.
K-Means	Clustering-based control	Map similar runtime states to representative rates	Recent latency, throughput, and error features grouped into clusters	Moderate	Useful middle ground between static thresholds and full reinforcement learning.
Bandit	RL action-value control	Choose the rate with the best immediate observed payoff	Short-horizon reward from recent behavior	Moderate	Strong on noisy systems where low-overhead adaptation is valuable.
SARSA	RL value learning	Learn state-action values on-policy while exploring	State bins derived from runtime metrics plus current action	High	Most exploratory RL method in this project; useful when controlled exploration helps.
Q-Learning	RL value learning	Learn long-run state-action value off-policy	State bins derived from runtime metrics and delayed reward	High	Strongest option when enough decisions and stable feedback are available.

**Figure 7:** Project-specific conceptual ladder from fixed-rate baselines to value-learning reinforcement-learning controllers. This is not a measured ranking; it summarizes relative modeling and operational complexity.

9 Discussion

9.1 What the project shows overall

The strongest project-level conclusion is that controller choice must be matched to the system. Q-Learning is strongest when the environment supplies rich and stable feedback, as in Train-Ticket. Rule-based control remains highly competitive when latency can be improved through transparent threshold logic, as in the Timescale system. Bandit becomes attractive when the system is operationally noisier, as in Petclinic.

Grafana QuickPizza adds a practical secondary conclusion: the controller-family comparison and runtime-change workflow remain usable on a fourth, Grafana-centered demo stack.

9.2 Operational guidance

- Use **Q-Learning** when long decision horizons and stable telemetry make long-run value learning realistic.
- Use **Bandit** when quick, low-overhead adaptation is needed and the environment is noisy or sparse.

- Use **Rule** when transparency, predictability, and ease of explanation are more important than model sophistication.
- Use **K-Means** when a moderate data-driven controller is preferred over manual thresholds.
- Treat **SARSA** as the most exploratory reinforcement-learning option in this project, useful when controlled exploration is acceptable.

9.3 Reward should not dominate the final interpretation

Reward was necessary to drive the reinforcement-learning controllers, but it cannot be allowed to dominate the final report. The Timescale results demonstrate why: the reward signal can look favorable even when the runtime outcome is poor. For that reason, the final synthesis prioritizes latency, QPS, error rate, and data quality over reward in the main findings.

9.4 Practical usefulness for engineers and operators

- If the system is stable and feedback is rich, Q-Learning is a credible choice because it can exploit longer decision horizons.
- If transparency and ease of explanation matter, Rule-based control remains attractive and can still win on important metrics.
- If the environment is noisy or sparse, Bandit offers a useful middle ground between adaptability and operational simplicity.
- If the goal is controller selection rather than algorithm advocacy, the study provides a practical basis for matching controller family to system behavior.

9.5 System-specific takeaways

Train-Ticket is the strongest setting for learning-oriented control because its traffic and decision horizon are stable enough to reward longer-term value learning. The Timescale application shows the opposite lesson: transparent heuristic control can still win on a meaningful top-line metric even when more sophisticated controllers are available. Petclinic demonstrates that noisy business-style microservice environments can favor simpler RL control such as Bandit over deeper value learning. QuickPizza broadens the study into a Grafana-centered observability stack and shows that the same overall controller taxonomy remains usable even when the surrounding telemetry platform differs substantially from the other applications.

Taken together, these four cases support a practical deployment rule: choose the controller family that fits the behavior of the system and the quality of the feedback loop, not the controller that looked best in a different environment. This is the central reason the paper treats adaptive tracing as a design-space problem rather than an algorithm race.

10 Threats to Validity and Limitations

- **Unequal evidence quality:** Train-Ticket remains the strongest result set overall. Petclinic and QuickPizza are more sensitive to sparse or noisy decision windows than Train-Ticket.
- **Reward comparability:** reward functions differ across systems, so cross-system reward comparisons are contextual rather than absolute.
- **Scenario heterogeneity:** the exact fault or degradation mechanism differs by application because the systems expose different control surfaces.
- **Different telemetry backends:** the systems use different data paths and observability stacks, which is realistic but introduces methodological heterogeneity.
- **Limited repetition:** the study uses completed matrices rather than repeated statistical replications for every scenario-policy combination.

11 Conclusion

This project evaluated adaptive tracing across four microservice systems through a controller-comparison study,

a runtime-change study, and a taxonomy synthesis. The combined evidence shows that reinforcement-learning-based adaptive tracing is consistently competitive but not universally dominant: Q-Learning leads Train-Ticket, Rule leads Timescale final latency, Bandit leads Petclinic final runtime metrics, and QuickPizza confirms that the same controller families remain viable on a Grafana-centered observability stack. The runtime-change analysis shows that controllers respond differently to degradation patterns rather than exhibiting one common adaptation profile. The taxonomy synthesis shows that adaptive tracing is best understood as a family of control strategies, not as a single technique.

The final engineering conclusion is direct: **there is no single best adaptive tracing controller for all microservice systems**. The appropriate choice depends on the stability of the runtime signal, the need for operational transparency, and the quality of the feedback available for learning.

The broader practical conclusion is equally important: the usefulness of adaptive tracing lies in giving teams a structured way to balance observability value against telemetry cost under realistic runtime conditions. The contribution of this paper is therefore both empirical and operational: it provides evidence, translates that evidence into controller-selection guidance, and validates that the same workflow can be applied across four distinct microservice environments.

References and Readings Consulted

References

- [1] Sigelman, B. H., Barroso, L. A., Burrows, M., Stephenson, P., Plakal, M., Beaver, D., Jaspan, S., and Shanbhag, C. *Dapper, a Large-Scale Distributed Systems Tracing Infrastructure*. Google technical report, 2010.
- [2] Fonseca, R., Porter, G., Katz, R. H., Shenker, S., and Stoica, I. *X-Trace: A Pervasive Network Tracing Framework*. NSDI, 2007.
- [3] Kaldor, J., Mace, J., Bejda, M., Gao, E., Kuropatwa, W., O'Neill, J., Ong, K., Schaller, B., Shan, P., Viscomi, B., Venkataraman, V., Veeraraghavan, K., Werner, K., and Zhou, Y. *Canopy: An End-to-End Performance Tracing and Analysis System*. SOSP, 2017.
- [4] Mace, J., Roelke, R., and Fonseca, R. *Pivot Tracing: Dynamic Causal Monitoring for Distributed Systems*. SOSP, 2015.
- [5] FudanSELab. *Train Ticket: A Benchmark Microservice System*. GitHub repository. <https://github.com/FudanSELab/train-ticket>. Consulted during deployment and architecture review. Accessed June 22, 2026.
- [6] OpenTelemetry Project. *OpenTelemetry Demo (Astronomy Shop)*. GitHub repository. <https://github.com/open-telemetry/opentelemetry-demo>. Consulted during deployment, instrumentation, and scenario review. Accessed June 22, 2026.
- [7] OpenTelemetry Project. *OpenTelemetry Demo Documentation*. <https://opentelemetry.io/docs/demo/>. Consulted during deployment and observability-stack review. Accessed June 22, 2026.
- [8] Spring Petclinic Community. *Spring Petclinic Microservices*. GitHub repository. <https://github.com/spring-petclinic/spring-petclinic-microservices>. Consulted during deployment and architecture review. Accessed June 22, 2026.
- [9] Spring Petclinic Community. *The Spring PetClinic Community*. <https://spring-petclinic.github.io/>. Consulted during project background and system-context review. Accessed June 22, 2026.
- [10] Grafana Labs. *QuickPizza*. GitHub repository. <https://github.com/grafana/quickpizza>. Consulted during deployment, observability-stack setup, and experiment integration. Accessed June 22, 2026.
- [11] OpenTelemetry Project. *OpenTelemetry Documentation and Demo Materials*. Online documentation consulted during instrumentation and architecture review. Accessed May 18, 2026.
- [12] Jaeger Project. *Jaeger Documentation*. Online documentation consulted during the tracing-tooling review. Accessed May 18, 2026.
- [13] Zipkin Project. *Zipkin Documentation*. Online documentation consulted for the Petclinic tracing backend. Accessed May 18, 2026.
- [14] *An Adaptive Logging System (ALS): Enhancing Software Logging with Reinforcement Learning Techniques*. Supplemental project reading consulted during the literature review.

Additional Readings Consulted

- *An Adaptive Logging System (ALS): Enhancing Software Logging with Reinforcement Learning Techniques*.
- Zhang et al., *NSDI 2023 paper consulted during the observability literature review*.
- *review.pdf* local survey reading consulted during the report framing stage.
- *ArXiv reading stored as https://arxiv.org/pdf/2509.13852.pdf* consulted during the broader adaptive observability review.